MBLP131 · Hafta 5 · Ders Notları

İşletim Sistemleri Temelleri

İşlemci Zamanlama Algoritmaları ve Süreç Eşzamanlaması

Sırada kim çalışacak ve iş parçacıkları veriyi bozmadan nasıl paylaşır?

Öğretim üyesi: Dr. Öğr. Üyesi Hasan Oğuz
Elektronik Haberleşme Teknolojisi Program Başkanı, Meslek Yüksekokulu
İstanbul Okan Üniversitesi

Bu haftanın sonunda yapabilmeniz gerekenler

1. Bir iş kümesinin FCFS, SJF, SRTF, öncelikli ve döngüsel (Round Robin) zamanlama altındaki Gantt şemasını çizmek; her sürecin bekleme, dönüş ve yanıt süresini hesaplamak.
2. Konvoy etkisini, SJF'nin ortalama bekleme süresini neden en aza indirdiğini ve bir sonraki CPU patlamasının üstel ortalamayla nasıl tahmin edildiğini açıklamak.
3. Açlığı (starvation) ve yaşlandırmayı (aging), döngüsel zamanlamada zaman dilimi seçimindeki ödünleşimi açıklamak.
4. Çok düzeyli geri beslemeli kuyrukları ve Linux'un kullandığı adil zamanlayıcıyı genel hatlarıyla anlatmak.
5. Bir yarış durumunu `counter++` işleminin oku–ekle–yaz dizisiyle açıklamak; kritik bölge problemini ve üç gereksinimini ifade etmek; Peterson çözümünü, atomik komutları, muteks kilitlerini ve semaforları anlatmak.
6. Sınırlı tampon problemini semaforlarla çözmek ve basit bir kilitlenmeyi tanımak.
7. Bir yarışı ve çözümünü gösteren küçük `pthreads` programlarını okumak, derlemek ve çalıştırmak.

İçindekiler

1 Tekrar ve Sürekli Örneğimiz 1

2 İlk Gelen İlk Hizmet Alır (FCFS) 2

3 En Kısa İş Önce Zamanlaması 3

4 Öncelikli Zamanlama 5

5 Döngüsel Zamanlama (Round Robin) 7

6 Çok Düzeyli Kuyruklar ve Gerçek Zamanlayıcılar (Genel Bakış) 9

7 Yarış Durumları 10

8 Kritik Bölge Problemi 12

9 Peterson Çözümü 13

10 Donanım Desteği: Atomik Komutlar 13

11 Muteks Kilitleri 14

12 Semaforlar 14

13 Sınırlı Tampon Problemi 15

14 Bir Uyarı: Kilitlenme 16

15 Uygulama: C'de İş Parçacıkları, Yarışlar ve Kilitler 16

16 Özet 19

17 Sözlük 19

18 Kendiniz Deneyin 20

19 Tekrar Soruları 20

20 İleriye Bakış: 6. Hafta 21

1 Tekrar ve Sürekli Örneğimiz

Geçen hafta hazır kuyruğundan bir süreç seçen **işlemci zamanlayıcısı** (CPU scheduler) ile işlemciyi ona gerçekten veren **dağıtıcıyı** (dispatcher) tanıdık (bağlam değişimi, kullanıcı moduna geçiş, doğru komuta atlama). Zamanlama **kesintisiz** (non-preemptive; süreç sonlanana ya da G/Ç için bloklanana kadar işlemciyi tutar) ya da **kesintili** (preemptive; işletim sistemi işlemciyi, tipik olarak bir zamanlayıcı kesmesinde ya da daha önemli bir süreç hazır olduğunda, geri alabilir) olabilir. Bir planı aşağıdaki ölçütlerle değerlendiririz; her süre tek bir süreç için ölçülür:

$$T_{\text{dönüş}} = T_{\text{bitiş}} - T_{\text{varış}},$$
$$T_{\text{bekleme}} = T_{\text{dönüş}} - T_{\text{patlama}},$$
$$T_{\text{yanıt}} = T_{\text{ilk çalışma}} - T_{\text{varış}}.$$

Bekleme süresi, bir sürecin *hazır kuyruğunda* geçirdiği toplam süredir. Kesintisiz bir algoritmada süreç tek parça hâlinde çalıştığından bekleme süresi yanıt süresine eşittir. İşlemci kullanımı ve iş çıkarma hızı (throughput) olabildiğince yüksek; dönüş (turnaround), bekleme (waiting) ve yanıt (response) süreleri olabildiğince düşük olmalıdır.

1.1 Her algoritma için tek bir iş kümesi

Algoritmaları adil biçimde karşılaştırmak için 1. bölüm boyunca *aynı* dört süreci kullanıyoruz. Tüm süreler milisaniye (ms) cinsindendir.

Süreç	Varış zamanı	CPU patlaması	Öncelik
P1	0	7	3
P2	2	4	1
P3	4	1	4
P4	5	4	2

Kurallar (ders kitapları farklı olduğundan bunları her sınav yanıtında belirtin):

- Bağlam değişimi süresi 0 alınır; geçiş sayısını ayrıca sayarız.
- Eşitlikler önce varış zamanına, sonra süreç adına göre bozulur.
- *Küçük* öncelik numarası *yüksek* öncelik demektir.
- Döngüsel zamanlamada, bir zaman dilimi bittiği anda gelen süreç, hazır kuyruğuna kesilen süreçten *önce* girer.

Toplam CPU işi $7+4+1+4=16$ ms'dir ve işlemci hiç boş kalmaz; bu yüzden aşağıdaki her plan $t=16$'da biter. **Gantt şeması**, kutulara bölünmüş bir zaman çizgisidir; her kutu, o aralıkta işlemciyi hangi sürecin kullandığını gösterir.

Bu notlardaki ve slaytlardaki her sayı, `code/` klasöründeki kısa bir Python programı, `sched_check.py` ile hesaplandı. Tüm Gantt şemalarını ve tabloları yazdırmak için `python3 sched_check.py` komutunu çalıştırın ya da kendi iş kümenizi denemek için programdaki `JOBS` listesini değiştirin.

2 İlk Gelen İlk Hizmet Alır (FCFS)

En basit politika: hazır kuyruğu bir FIFO (first-in, first-out; ilk giren ilk çıkar) kuyruğudur ve önce gelen süreç işlemciyi alır, bitene ya da bloklanana kadar tutar. FCFS (First-Come, First-Served) kesintisizdir.

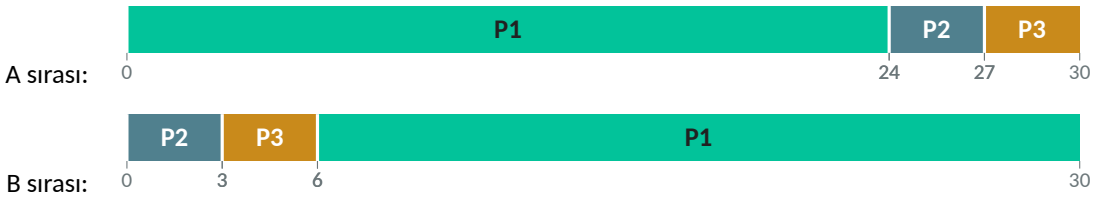


Süreç	Varış	Patlama	Bitiş	Dönüş	Bekleme	Yanıt
P1	0	7	7	7	0	0
P2	2	4	11	9	5	5
P3	4	1	12	8	7	7
P4	5	4	16	11	7	7
Ortalama				8.75	4.75	4.75

Örneğin P3, $t = 4$'te gelir, $t = 11$'de başlar ve $t = 12$'de biter: $T_{\text{dönüş}} = 12 - 4 = 8$ ms ve $T_{\text{bekleme}} = 8 - 1 = 7$ ms. Ortalama bekleme süresi $(0 + 5 + 7 + 7)/4 = 4.75$ ms'dir.

2.1 Konvoy etkisi

FCFS "banka kuyruğu" anlamında adildir, ama ortalama bekleme süresi süreçlerin tesadüfen hangi sırayla geldiğine büyük ölçüde bağlıdır. Hepsi $t = 0$'da gelen, patlamaları 24, 3 ve 3 ms olan üç iş düşünün.



$$\text{A sırası:}\quad \overline{T}_{\text{bekleme}} = \frac{0 + 24 + 27}{3} = \frac{51}{3} = 17 \text{ ms},$$
$$\text{B sırası:}\quad \overline{T}_{\text{bekleme}} = \frac{0 + 3 + 6}{3} = \frac{9}{3} = 3 \text{ ms}.$$

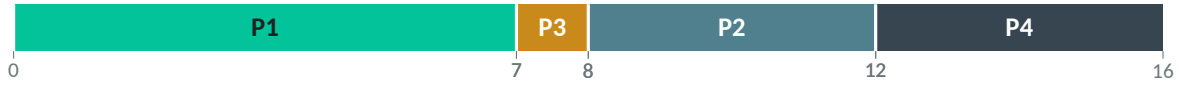
Aynı işler, aynı algoritma, neredeyse altı kat daha az bekleme. Çok sayıda kısa, G/Ç ağırlıklı süreç tek bir uzun, CPU ağırlıklı sürecin arkasında sıkıştığında, dağ yolunda yavaş bir kamyonun arkasındaki arabalar gibi bütün sistem yavaşlar. Buna **konvoy etkisi** (convoy effect) denir: kısa işler beklerken G/Ç aygıtları boş durur, sonra hepsi G/Ç yaparken işlemci boş durur.

3 En Kısa İş Önce Zamanlaması

Yukarıdaki B sırası daha iyi bir kural önerir: *en kısa işi önce çalıştır* (SJF, Shortest-Job-First). Aslında kural bütün işin değil, *bir sonraki CPU patlamasının* uzunluğuna bakar; bu yüzden daha doğru bir ad "en kısa sonraki CPU patlaması önce" olurdu.

3.1 Kesintisiz SJF

İşlemci her boşaldığında, sonraki patlaması en kısa olan hazır süreci seç; başlayan süreç sonuna kadar çalışır.



$t = 0$'da yalnızca P1 hazırdır, bu yüzden $t = 7$'ye kadar çalışır. $t = 7$'de hazır kuyruğunda P2 (4 ms), P3 (1 ms) ve P4 (4 ms) vardır; en kısası P3'tür. $t = 8$'de P2 ile P4 4 ms'de eşittir ve P2 önce gelmiştir.

Bekleme süreleri P1 0, P2 $12 - 2 - 4 = 6$, P3 $8 - 4 - 1 = 3$ ve P4 $16 - 5 - 4 = 7$ ms; dönüş süreleri 7, 10, 4 ve 11 ms'dir.

Ortalama bekleme süresi 4.75 ms'den (FCFS) 4 ms'ye düşer. Kalan zayıflığa dikkat edin: P1, $t = 0$'da yalnızca başka kimse olmadığı için başladı ve çok kısa P3 onu beklemek zorunda kaldı.

3.2 Kesintili SJF: en kısa kalan süre önce (SRTF)

Kesintili sürümde (SRTF, Shortest-Remaining-Time-First), her yeni süreç geldiğinde zamanlayıcı onun patlamasını çalışan sürecin *kalan* süresiyle karşılaştırır; yeni gelen daha kısaysa işlemciyi o alır.



- $t = 2$: P2 gelir ve 4 ms ister; P1'in hâlâ $7 - 2 = 5$ ms'si vardır. $4 < 5$ olduğundan P1 kesilir.
- $t = 4$: P3 gelir ve 1 ms ister; P2'nin hâlâ 2 ms'si vardır. P2 kesilir.
- $t = 5$: P3 biter. Kalan süreler: P2 2, P4 4, P1 5. Böylece P2 5–7, sonra P4 7–11, sonra P1 11–16 arasında çalışır.

Süreç	Varış	Patlama	Bitiş	Dönüş	Bekleme	Yanıt
P1	0	7	16	16	9	0
P2	2	4	7	5	1	0
P3	4	1	5	1	0	0
P4	5	4	11	6	2	2
Ortalama				7	3	0.5

Kesintili zamanlamada bir süreç birkaç parça hâlinde çalışabilir; bu yüzden bekleme süresini boşlukları toplayarak değil formülle hesaplarız: P1 16'da biter, dolayısıyla $T_{\text{bekleme}}(\text{P1}) = (16 - 0) - 7 = 9$ ms (sağlama: P1 2'den 11'e kadar, yani 9 ms bekledi). Ortalama bekleme süresi $(9 + 1 + 0 + 2)/4 = 3$ ms ile günün en iyi değeridir; kısa işlerin hızının bedelini uzun iş P1 öder.

3.3 En kısa önce neden en iyisidir?

SJF'nin en iyiliği (optimality)

Tüm işler aynı anda hazırsa, onları patlama uzunluğuna göre artan sırayla çalıştırmak olası en küçük ortalama bekleme süresini verir. İşler farklı zamanlarda geliyorsa en küçük değeri SRTF verir (bağlam değişimleri bedavaysa).

İspat bir *yer değiştirme argümanıdır* (exchange argument). Daha uzun bir L işinin, daha kısa bir S işinden ($L > S$) hemen önce, bir t_0 anından başlayarak çalıştığı herhangi bir planı düşünün. İkisinin yerini değiştirin:

- Çiftten önceki her iş eskisi gibi aynı anda başlar; çiftten sonraki her iş de aynı anda başlar, çünkü $L + S$ değişmez.
- Çiftin içinde bekleme süreleri t_0 (L için) ve $t_0 + L$ (S için) idi; yer değiştirmeden sonra t_0 ve $t_0 + S$ olur.

Toplam bekleme süresi dolayısıyla $L - S > 0$ kadar azalır. Uzun bir işin kısa bir işten önce geldiği her plan iyileştirilebilir; bu yüzden en kısa önce sırası en iyisidir. Konvoy örneğinde B sırası tam olarak en kısa önce sırasıdır.

3.4 Püf noktası: bir sonraki CPU patlamasını tahmin etmek

SJF, süreç çalışmadan *önce* bir sonraki CPU patlamasının uzunluğunu bilmeyi gerektirir. Uzun vadeli toplu iş zamanlamasında kullanıcılardan işlerinin çalışma süresini tahmin etmeleri istenebilirdi; işlemci zamanlamasında ise bunu bilmenin hiçbir yolu yoktur. İşletim sistemi, bir sonraki patlamanın son patlamalara benzeyeceğini varsayarak yalnızca *tahmin* edebilir. Standart tahmin yöntemi **üstel ortalamadır** (exponential average):

$$\tau_{n+1} = \alpha\, t_n + (1-\alpha)\,\tau_n, \qquad 0 \le \alpha \le 1,$$

burada t_n, n'inci patlamanın ölçülen uzunluğu, τ_n ise onun için yapılmış tahmindir. Özyinelemeyi açmak, adın nereden geldiğini gösterir:

$$\tau_{n+1} = \alpha t_n + (1-\alpha)\alpha t_{n-1} + (1-\alpha)^2 \alpha t_{n-2} + \cdots + (1-\alpha)^{n+1}\tau_0.$$

$1-\alpha < 1$ olduğundan eski patlamalar üstel olarak küçülen ağırlıklar alır. $\alpha = 0$ iken tahmin hiç değişmez (yakın geçmiş yok sayılır); $\alpha = 1$ iken yalnızca son patlama sayılır. Yaygın bir seçim $\alpha = 1/2$'dir.

Üstel ortalama, $\alpha = 1/2$, $\tau_0 = 10$ ms

Sürecin gerçek patlamaları $t_0, t_1, \cdots = 6, 4, 6, 4, 13, 13, 13$ ms'dir. İlk iki adım:

$$\tau_1 = \tfrac{1}{2}\cdot 6 + \tfrac{1}{2}\cdot 10 = 8 \text{ ms},$$
$$\tau_2 = \tfrac{1}{2}\cdot 4 + \tfrac{1}{2}\cdot 8 = 6 \text{ ms}.$$

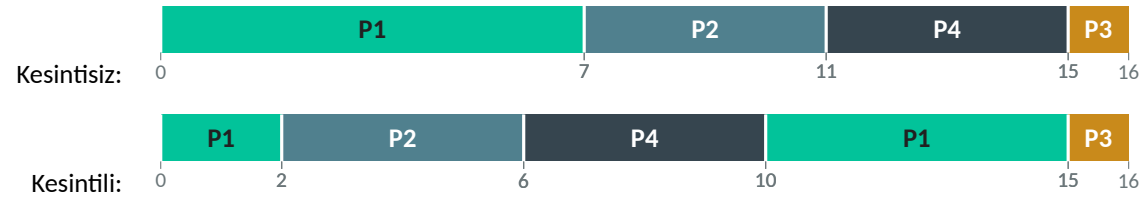
Aynı biçimde devam edersek:

n	0	1	2	3	4	5	6	7
gerçek patlama t_n (ms)	6	4	6	4	13	13	13	?
tahmin τ_n (ms)	10	8	6	6	5	9	11	12

Sürecin davranışı değiştiğinde (patlamalar 13 ms'ye sıçradığında) tahmin onu gecikmeyle izler: $5 \to 9 \to 11 \to 12$. Bu gecikme, rastgele dalgalanmaları yumuşatmanın bedelidir.

4 Öncelikli Zamanlama

Her sürece bir **öncelik** (priority) verilir ve işlemci en yüksek öncelikli hazır sürece gider. Bu derste (Linux'ta ve Silberschatz'da olduğu gibi) küçük numara yüksek öncelik demektir. SJF, önceliğin tahmin edilen patlama uzunluğu olduğu özel durumdur. Öncelikli zamanlama kesintisiz ya da kesintili olabilir.



Kesintili sürümde P2 (öncelik 1), $t = 2$'de gelir gelmez işlemciyi P1'den (öncelik 3) alır ve 6'ya kadar çalışır; sonra P4 (öncelik 2, 5'te gelmişti) 6–10 arasında çalışır; P1 10–15 arasında biter; P3 en son çalışır.

		Kesintisiz		Kesintili	
Süreç	**Öncelik**	**Bekleme**	**Yanıt**	**Bekleme**	**Yanıt**
P1	3	0	0	8	0
P2	1	5	5	0	0
P3	4	11	11	11	11
P4	2	6	6	1	1
Ortalama		5.5	5.5	5	3

Ortalama dönüş süreleri 9.5 ms (kesintisiz) ve 9 ms'dir (kesintili). Öncelikli zamanlama ortalamaları en aza indirmeye çalışmaz: *öneme* hizmet eder. Öncelikler *iç* (işletim sisteminin bellek ihtiyacı ya da G/Ç patlamalarının CPU patlamalarına oranı gibi ölçülebilir büyüklüklerden hesapladığı) ya da *dış* (dışarıdan belirlenen: işin önemi, onun için ödeme yapan birim) olabilir.

4.1 Açlık ve yaşlandırma

En düşük öncelikli P3 iki sürümde de 11 ms bekledi. Yüksek öncelikli süreçlerin sürekli geldiği yoğun bir sistemde düşük öncelikli bir süreç *hiçbir zaman* çalışmayabilir. Buna **açlık** (starvation) ya da *süresiz bloklanma* (indefinite blocking) denir.

> **Altı yıl bekleyen iş**
>
> Sıkça anlatılan bir hikâyeye göre MIT'deki IBM 7094 1973'te kapatıldığında operatörler, 1967'de gönderilmiş ve hiç çalıştırılmamış düşük öncelikli bir süreç bulmuş. Her ayrıntısı doğru olsun ya da olmasın, sorunu çok iyi anlatır.

Standart çare **yaşlandırmadır** (aging): bir sürecin önceliğini, ne kadar uzun beklerse o kadar yükseltmek. Örneğin öncelikler 127 (en düşük) ile 0 (en yüksek) arasındaysa ve bekleyen bir süreç her saniye bir öncelik düzeyi kazanıyorsa, 127'den başlayan bir süreç bile en geç

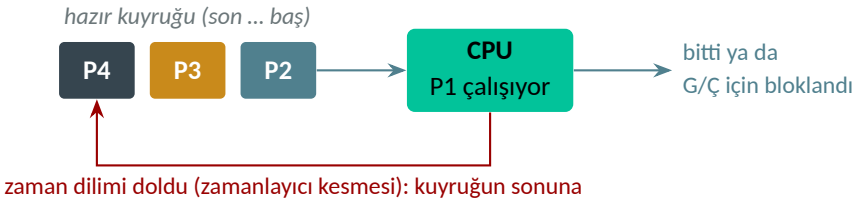
$$127 \times 1\ \text{s} = 127\ \text{s} = 2\ \text{dk}\ 7\ \text{s}$$

sonra 0 önceliğine ulaşır. Yaşlandırma "belki hiçbir zaman"ı "sınırlı bir süre içinde"ye çevirir. Aynı fikirle Bölüm 8'de *sınırlı bekleme* gereksinimi olarak yeniden karşılaşacağız.

5 Döngüsel Zamanlama (Round Robin)

Döngüsel zamanlama (Round Robin, RR), zaman paylaşımlı sistemler için tasarlanmış, kesintili bir FCFS'dir. **Zaman dilimi** (time quantum ya da time slice) adı verilen küçük bir zaman birimi q tanımlanır; tipik olarak 10–100 ms. Hazır kuyruğu dairesel bir FIFO kuyruğu olarak ele alınır:

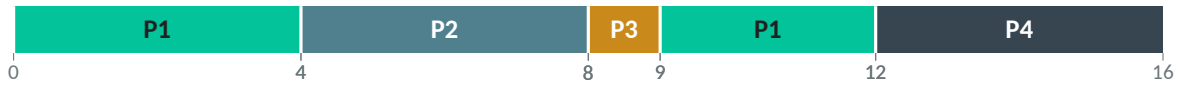
1. Zamanlayıcı kuyruğun başındaki süreci alır ve q sonra kesme üretecek bir zamanlayıcı kurar.
2. CPU patlaması q dolmadan biterse (süreç sonlanır ya da G/Ç için bloklanır), süreç işlemciyi kendiliğinden bırakır ve sıradaki hemen başlar.
3. Önce zamanlayıcı çalarsa süreç kesilir ve hazır kuyruğunun *sonuna* konur.



Şekil 1: Döngüsel zamanlama.

Hazır kuyruğunda n süreç varsa her biri işlemci zamanının $1/n$'ini en çok q uzunluğundaki parçalar hâlinde alır ve hiçbir süreç bir sonraki sırasından önce $(n-1)\,q$'dan fazla beklemez. RR'yi yanıt süresi açısından iyi yapan bu sınırdır.

5.1 Çözümlü örnek: $q = 4$ ms



Hazır kuyruğunu izleyin (baş solda):

- $t = 0$: yalnızca P1 var; tam bir zaman dilimi, 0–4 arasında çalışır.
- $t = 4$: P2 (2'de geldi) ve P3 (4'te geldi) kuyrukta; P1 sona katılır (kuralımız: tam 4'te gelen P3, P1'den önce girer). Kuyruk: P2, P3, P1.
- $t = 8$: P2 4 ms'sini kullandı ve bitti. P4 5'te geldi. Kuyruk: P3, P1, P4.
- $t = 9$: P3 1 ms sonra bitti. P1'in 3 ms daha işi var ve 12'de biter; sonra P4 12–16 arasında çalışır.

Süreç	Varış	Patlama	Bitiş	Dönüş	Bekleme	Yanıt
P1	0	7	12	12	5	0
P2	2	4	8	6	2	2
P3	4	1	9	5	4	4
P4	5	4	16	11	7	7
Ortalama				8.5	4.5	3.25

5.2 Çözümlü örnek: $q = 2$ ms



Zaman	Olay	Sonraki hazır kuyruğu (baş önce)
0	P1 başlar	—
2	P2 geldi; P1'in zaman dilimi doldu	P2, P1
4	P3 geldi; P2'nin zaman dilimi doldu	P1, P3, P2
6	P4 5'te geldi; P1'in zaman dilimi doldu	P3, P2, P4, P1
7	P3 biter (1 ms'lik patlama)	P2, P4, P1
9	P2 biter	P4, P1
11	P4'ün zaman dilimi doldu	P1, P4
13	P1'in zaman dilimi doldu	P4, P1
15	P4 biter; P1 son 1 ms'sini çalışır	—

Bitiş zamanları P1 16, P2 9, P3 7, P4 15'tir; dolayısıyla bekleme süreleri 9, 3, 2 ve 6 ms, yanıt süreleri 0, 0, 2 ve 4 ms'dir (ortalamalar: bekleme 5, dönüş 9, yanıt 1.5 ms).

$q = 4$ ile karşılaştırıldığında ortalama yanıt süresi 3.25'ten 1.5 ms'ye iyileşir, ama ortalama bekleme süresi 4.5'ten 5 ms'ye çıkar ve bağlam değişimi sayısı 4'ten 8'e, iki katına çıkar.

5.3 Zaman dilimini seçmek

Zaman dilimi q	Ort. bekleme (ms)	Ort. yanıt (ms)	Bağlam değişimi
1 ms	5.5	0.75	14
2 ms	5	1.5	8
4 ms	4.5	3.25	4
8 ms	4.75	4.75	3

- q **çok büyükse:** RR, FCFS'ye dönüşür. İş kümemizdeki her patlamadan uzun olan $q = 8$ ms ile plan FCFS ile *aynıdır*.
- q **çok küçükse:** işlemci zamanının büyük bir kısmı geçişe gider. Her geçiş s kadar sürüyorsa, q kadar yararlı işin her dilimi bir geçişe mal olur ve kaybedilen işlemci zamanı payı

$$\frac{s}{q + s}$$

olur. (Bilerek kötümser seçilmiş) $s = 0{,}1$ ms'lik bir geçiş maliyetiyle bu, $q = 1$ ms için %9,1, $q = 4$ ms için %2,4, $q = 10$ ms için %1,0 ve $q = 100$ ms için %0,1 eder.

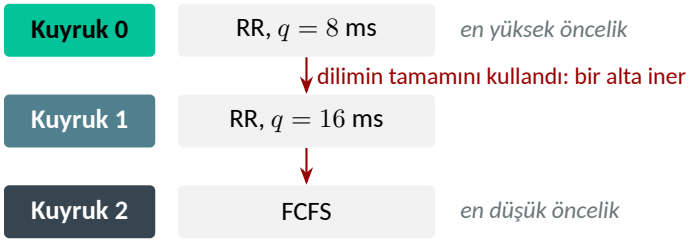
Gerçek bağlam değişimleri birkaç mikrosaniye sürer; buna bir de dolaylı maliyet eklenir: yeni zamanlanan süreç için önbellekler ve TLB "soğuktur". Yaygın bir pratik kural, q'yu CPU patlamalarının yaklaşık %80'i q'dan kısa olacak biçimde seçmektir; böylece etkileşimli süreçler patlamalarını genellikle tek bir zaman dilimi içinde bitirir.

6 Çok Düzeyli Kuyruklar ve Gerçek Zamanlayıcılar (Genel Bakış)

6.1 Çok düzeyli kuyruk ve çok düzeyli geri beslemeli kuyruk

Gerçek sistemler birkaç algoritmayı birleştirir. **Çok düzeyli kuyruk** (multilevel queue) zamanlayıcısında hazır kuyruğu birkaç kuyruğa bölünür; örneğin RR ile zamanlanan *ön plan* (etkileşimli) süreçleri ve FCFS ile zamanlanan *arka plan* (toplu iş) süreçleri, kuyruklar arasında sabit öncelikle. Süreç kalıcı olarak kendi kuyruğunda kalır.

Çok düzeyli geri beslemeli kuyruk (multilevel feedback queue, MLFQ) ise süreçlerin davranışlarına göre kuyruklar arasında yer değiştirmesine izin verir. Klasik bir örnekte üç kuyruk vardır:



Şekil 2: Üç düzeyli bir geri beslemeli kuyruk.

Yeni bir süreç kuyruk 0'a girer. Patlamasını 8 ms içinde bitirmezse kuyruk 1'e iner; yine 16 ms'lik zaman diliminin tamamını kullanırsa kuyruk 2'ye iner. Kuyruk 1 yalnızca kuyruk 0 boşken çalışır, ve bu böyle sürer. Böylece kısa, etkileşimli patlamalar yüksek öncelikte biterken uzun, CPU ağırlıklı süreçler dibe çöker. Düşük bir kuyrukta çok uzun bekleyen bir süreç yeniden yukarı taşınabilir: bu, yaşlandırmadır. MLFQ, patlama uzunluklarını önceden *bilmeden*, her sürecin nasıl davrandığını gözleyerek SJF'ye yaklaşır. En genel yöntemdir ve ayarlaması en zor olanıdır.

6.2 Linux ne yapıyor?

Linux sabit bir zaman dilimi kullanmaz. Normal süreçler için zamanlayıcısı *adaleti* hedefler: çalışmaya hazır her görev, işlemciden **ağırlığıyla** (weight) orantılı bir pay almalıdır.

- Çekirdek 2.6.23'ten (2007) 6.5'e kadar bu, **Completely Fair Scheduler (CFS)** idi. Her görevin bir *sanal çalışma süresi* vardır: işlemci süresinin ağırlığına bölümü; CFS her zaman sanal çalışma süresi en küçük olan görevi çalıştırır.
- Çekirdek 6.6'dan (2023) beri varsayılan zamanlayıcı **EEVDF**'dir (*Earliest Eligible Virtual Deadline First*). Adalet hedefini korur, ama ek olarak her göreve bir sanal son tarih verir; böylece düşük gecikme isteyen görevler daha erken çalışabilir.
- Ağırlık, -20 (en çok işlemci) ile $+19$ (en az işlemci) arasında, varsayılanı 0 olan **nice değerinden** gelir. Her nice adımı ağırlığı yaklaşık 1.25 kat değiştirir; nice 0'ın ağırlığı 1024, nice 10'unki 110'dur.
- Gerçek zamanlı zamanlama sınıfları (`SCHED_FIFO`, `SCHED_RR`, öncelikler 1–99) her zaman normal görevlerden önce çalışır.

nice etkisini gerçek bir çekirdekte ölçmek

Sonsuz bir döngünün (`for(;;) x++;`, program `code/cpu_hog.c`) iki kopyası aynı işlemciye sabitlenir; biri nice 0'da, öteki nice 10'da. Beklenen paylar şunlardır:

$$\frac{1024}{1024+110} \approx 90.3\%, \qquad \frac{110}{1024+110} \approx 9.7\%, \qquad \text{oran } \frac{1024}{110} \approx 9.31.$$

Derleme makinemizde (Linux 6.18, yani EEVDF) 10 s sonra ölçülen:

```
$ taskset -c 1 ./cpu_hog &
$ taskset -c 1 nice -n 10 ./cpu_hog &
$ sleep 10; ps -o pid,ni,cls,pcpu,comm -C cpu_hog
  PID  NI CLS %CPU COMMAND
 7150   0  TS 88.0 cpu_hog
 7151  10  TS  9.4 cpu_hog
```

Ölçülen $88.0/9.4 \approx 9.4$ oranı tahminle iyi uyuşur. (İki yüzdenin toplamı 100 etmez, çünkü `ps` her sürecin başladığı andan beri olan ortalamayı bildirir ve iki süreç tam olarak aynı anda başlamadı. `CLS` TS "time sharing", yani zaman paylaşımı anlamına gelir; normal sınıftır.)

6.3 İş kümemiz üzerinde karşılaştırma

Algoritma	Kesintili	Ort. bekleme	Ort. dönüş	Ort. yanıt	Geçiş
FCFS	hayır	4.75	8.75	4.75	3
SJF	hayır	4	8	4	3
SRTF	evet	3	7	0.5	5
Öncelik	hayır	5.5	9.5	5.5	3
Öncelik	evet	5	9	3	4
RR, $q=4$	evet	4.5	8.5	3.25	4
RR, $q=2$	evet	5	9	1.5	8

Tüm süreler ms cinsindendir. Bu küçük iş kümesinde SRTF her sütunu kazanır, ama geleceği bilmesi gerekir ve uzun işleri aç bırakabilir. RR hiçbir bilgiye ihtiyaç duymaz ve her sürece $(n-1)q$ içinde sıra gelmesini garanti eder. FCFS en basitidir. Öncelikli zamanlama ortalamaları değil önemi gözetir. Başka bir iş kümesi tablonun ortasındaki sırayı değiştirebilir, ama geçişler bedavayken en küçük ortalama bekleme süresini her zaman SRTF verir.

7 Yarış Durumları

1. bölümde zamanlayıcı süreçleri istediği an kesiyordu. Bağımsız süreçler için bu zararsızdır. Ama bir sürecin iş parçacıkları genel (global) değişkenlerini ve öbek belleğini (heap) paylaşır (4. hafta); bu durumda bir kesme ya da ikinci bir işlemci ortak veriyi bozabilir.

7.1 `counter++` tek bir adım değildir

İki iş parçacığının artırdığı genel bir `counter` değişkeni düşünün. C'de bu tek bir deyimdir, ama işlemci üç komut yürütür. Bu, x86-64 üzerinde `race.c` programımız için `gcc -O2 -S` komutunun gerçek çıktısıdır:

```
movq  counter(%rip), %rax     # 1. OKU:  counter'ı bellekten rax yazmacına
addq  $1, %rax                # 2. EKLE: yazmacın içinde 1 ekle
movq  %rax, counter(%rip)     # 3. YAZ:  yazmacı belleğe geri yaz
```

`counter` değişkeni ortak bellekte durur, ama `rax` yazmacı her iş parçacığına özeldir: bir bağlam değişiminde değeri iş parçacığının denetim bloğuna kaydedilir ve daha sonra geri yüklenir. Üç komutun herhangi ikisi arasında bir geçiş olabilir ve çok işlemcili bir sistemde diğer iş parçacığı aynı anda başka bir işlemcide çalışabilir.

Şanssız bir iç içe geçme

`counter` 5 ile başlar. A ve B iş parçacıkları `counter++` işlemini birer kez yürütür; dolayısıyla doğru sonuç 7'dir.

Adım	İş parçacığı A	İş parçacığı B	A'nın rax'ı	B'nin rax'ı	counter
1	oku		5	–	5
2		oku	5	5	5
3	1 ekle		6	5	5
4		1 ekle	6	6	5
5	yaz		6	6	6
6		yaz	6	6	6 (7 olmalıydı)

B'nin yazması A'nınkinin üstüne yazar: bir artırma kaybolur. Başka sıralar (örneğin önce A'nın üç adımı, sonra B'nin üç adımı) doğru sonucu, 7'yi verir. Sonuç zamanlamaya bağlıdır.

7.2 Tanım

Yarış durumu

Birden çok iş parçacığı ya da süreç aynı veriye eşzamanlı eriştiğinde, en az biri ona yazdığında ve sonuç erişimlerin hangi sırayla gerçekleştiğine bağlı olduğunda bir **yarış durumu** (race condition) oluşur.

Gündelik örnekler: iki ATM aynı anda aynı hesaptan para çeker ve ikisi de eski bakiyeyi okur; iki rezervasyon sunucusu da "bir koltuk kaldı" görür ve ikisi de onu satar. Çekirdeğin içinde: iki süreç aynı anda `fork()` çağırırsa ve çekirdeğin `next_available_pid` değişkeni korunmuyorsa, iki çocuk süreç aynı süreç kimliğini alabilir. Yarış durumları özellikle sinsidir, çünkü *zamanlama hatalarıdır*: bir program bin testi geçip bir sonraki çalıştırmada ya da yalnızca daha hızlı bir makinede başarısız olabilir. Tasarım yoluyla önlenmeleri gerekir.

8 Kritik Bölge Problemi

Kritik bölge (critical section), bir iş parçacığının ortak veriye (bir değişkene, bir listeye, bir dosyaya) eriştiği kod parçasıdır. **Kritik bölge problemi**, iş parçacıklarının kullanabileceği ve hiçbir ikisinin kritik bölgelerini aynı anda yürütmemesini sağlayan bir protokol tasarlamaktır. Her iş parçacığı şu yapıyı izler:

```
while (true) {
    /* giriş bölümü:  izin iste                    */
        /* kritik bölge: ortak veriyi kullan       */
    /* çıkış bölümü:  işin bittiğini duyur         */
    /* kalan bölüm:   geri kalan her şey           */
}
```

Doğru bir çözüm üç gereksinimi karşılamalıdır:

1. **Karşılıklı dışlama** (mutual exclusion). Bir iş parçacığı kritik bölgesini yürütüyorsa başka hiçbir iş parçacığı kendi kritik bölgesini yürütemez.
2. **İlerleme** (progress). Kritik bölgesinde hiçbir iş parçacığı yoksa ve bazı iş parçacıkları girmek istiyorsa, sıradakinin kim olacağına yalnızca bu iş parçacıkları karar verir ve karar sonsuza dek ertelenemez. (Kalan bölümündeki iş parçacıklarının veto hakkı yoktur.)
3. **Sınırlı bekleme** (bounded waiting). Bir iş parçacığı girmek için istekte bulunduktan sonra, diğer iş parçacıklarının ondan önce kritik bölgelerine girme sayısının bir sınırı vardır. Başka bir deyişle: açlık yok.

Her iş parçacığının sıfırdan büyük bir hızla çalıştığını varsayarız, ama iş parçacıklarının göreli hızları ya da işlemci sayısı hakkında *hiçbir* varsayım yapmayız.

Her gereksinim "ucuz" bir çözümü dışarıda bırakır: *kimseyi asla içeri alma* karşılıklı dışlamayı sağlar ama ilerlemeyi sağlamaz; *katı sırayla geçiş* (A, B, A, B, …) B girmek istemediğinde ilerlemeyi ihlal eder; her zaman A'yı kayıran bir kilit B için sınırlı beklemeyi ihlal edebilir.

8.1 Neden kesmeleri kapatmıyoruz?

Tek işlemcide, çalışan bir iş parçacığı işlemciyi yalnızca bir kesmeyle (ya da kendiliğinden bloklanarak) kaybeder. Dolayısıyla bir iş parçacığı kritik bölgesinin başında kesmeleri kapatıp sonunda yeniden açabilirdi; bağlam değişimi yok, yarış yok. Erken UNIX çekirdekleri tam olarak bunu yapıyordu. Ama:

- **Çok işlemcili** bir sistemde diğer işlemciler çalışmayı sürdürür ve ortak veriye yine erişebilir. Kesmeleri *tüm* işlemcilerde kapatmak her birine bir mesaj göndermeyi gerektirir; bu yavaştır ve bütün makineyi durdurur.
- Kesmeleri kapatmak **ayrıcalıklı bir komuttur** (2. hafta): kullanıcı programları bunu yapamaz ve yapmalarına izin verilmemelidir, çünkü kesmeleri hiç yeniden açmayan bir program makineyi ele geçirirdi.
- Uzun bir kritik bölge boyunca saate ve G/Ç aygıtlarına hizmet verilmez.

Linux, çekirdeğin içinde birkaç komutluk süre için *yerel* işlemcide kesmeleri hâlâ kapatır, ama bunu her zaman işlemciler arasında çalışan bir kilitle birlikte yapar.

9 Peterson Çözümü

Peterson çözümü (Gary Peterson, 1981), iki iş parçacığı, P_0 ve P_1 için klasik, yalnızca yazılıma dayalı bir çözümdür. İki ortak değişken kullanır:

```
int  turn;       /* bekleme sırası kimde?            */
bool flag[2];    /* flag[i] == true: Pi girmek istiyor */

/* i iş parçacığının kodu;  j = 1 - i diğer iş parçacığı */
while (true) {
    flag[i] = true;             /* girmek istiyorum ...    */
    turn = j;                   /* ... ama önce sen geç    */
    while (flag[j] && turn == j)
        ;                       /* meşgul bekleme */
    /* kritik bölge */
    flag[i] = false;            /* işim bitti */
    /* kalan bölüm */
}
```

Neden çalışır?

- *Karşılıklı dışlama*. İki iş parçacığı da kritik bölgede olsaydı iki bayrak da true olurdu. O zaman her biri bekleme döngüsünden `turn` kendi numarasına eşit olduğu için çıkmış olmalıydı; oysa `turn` yalnızca tek bir değer tutar. Somut olarak: `turn` değişkenine *en son* yazan iş parçacığı sırayı vermiştir ve beklemek zorundadır.
- *İlerleme*. Bir iş parçacığı yalnızca diğeri girmek istiyorsa *ve* sıra ondaysa bekler; diğeri kalan bölümündeyse `flag[j]` false'tur ve iş parçacığı hemen girer.
- *Sınırlı bekleme*. P_j çıkarken `flag[j] = false` yapar. Hemen yeniden girmek isterse `turn = i` yapar ve beklemek zorunda kalır. Dolayısıyla P_i, P_j'nin en çok bir girişini bekler.

Bir uyarı. Peterson çözümü *fikri* için gösterilir. Modern işlemcilerde yazıldığı hâliyle çalışması garanti değildir: daha hızlı çalışmak için işlemciler ve derleyiciler birbirinden bağımsız bellek erişimlerinin *sırasını değiştirebilir* (örneğin `flag[j]` okuması fiilen `flag[i]` yazmasından önce gerçekleşebilir). Doğru sürümler *bellek bariyerleri* (memory barriers) gerektirir. Uygulamada bir sonraki bölümün donanım desteğini kullanırız.

10 Donanım Desteği: Atomik Komutlar

Modern işlemciler, bir bellek sözcüğünü tek bir **atomik** (bölünmez) işlem olarak okuyup değiştiren özel komutlar sunar: arada başka hiçbir işlemci o sözcüğe erişemez. Açıklaması en kolay olanı `test_and_set`'tir. *Etkisi* C ile yazılabilir, ama donanımın onu tek bir adımda yürüttüğünü unutmayın:

```
bool test_and_set(bool *target) {   /* ATOMİK olarak yürütülür */
    bool old = *target;
    *target = true;
    return old;
}

bool lock = false;                  /* ortak; false = boş */

```

```
while (true) {
    while (test_and_set(&lock))
        ;                           /* dön: kilit zaten alınmış */
    /* kritik bölge */
    lock = false;                   /* bırak */
    /* kalan bölüm */
}
```

`test_and_set` `false` döndürürse kilit boştu ve aynı atomik adım onu zaten alınmış olarak işaretledi: kilit artık bu iş parçacığının. `true` döndürürse kilidi başka bir iş parçacığı tutuyordur ve yeniden deneriz. İki iş parçacığı asla ikisi birden `false` göremez, çünkü atomik işlemleri birbiri ardına yürütülür. Bu basit kilit karşılıklı dışlamayı ve ilerlemeyi sağlar; sınırlı bekleme biraz daha fazlasını gerektirir (örneğin bekleyen iş parçacıklarına geliş sırasıyla hizmet veren bir *bilet kilidi*, ticket lock).

Gerçek komutlar: x86'da `xchg` (atomik değiş tokuş) ve `lock cmpxchg` (**karşılaştır ve değiştir**, compare-and-swap, CAS: "sözcük hâlâ beklediğim değerdeyse onu yeni bir değerle değiştir"); ARM'da `ldxr/stxr` çifti. C11 standardı bu komutları taşınabilir fonksiyonlar olarak sunar; örneğin `atomic_flag_test_and_set()` ve `atomic_compare_exchange_strong()` fonksiyonları bu amaçla kullanılır.

11 Muteks Kilitleri

Uygulama programcıları atomik komutları nadiren doğrudan kullanır. İşletim sistemleri ve kütüphaneler onları iki işlemli bir **muteks kilidine** (mutex lock; *mutual exclusion* sözcüklerinden) sarar:

```
acquire(&m);       /* m boşalana kadar bekle, sonra al */
    /* kritik bölge */
release(&m);       /* geri ver */
```

`acquire` ve `release` çağrılarının kendileri de atomik olmalıdır; donanım komutları bunun içindir. POSIX threads'te bu işlemlerin adları `pthread_mutex_lock()` ve `pthread_mutex_unlock()`'tır.

Bir kilidi beklemenin iki yolu vardır:

- **Meşgul bekleme (döner kilit, spinlock).** Bekleyen iş parçacığı kilidi tekrar tekrar sınayarak döngüde döner. Bu işlemci zamanını boşa harcar, ama bağlam değişimi gerekmez. Döner kilitler, kritik bölgeler çok kısaysa çok işlemcili sistemlerde iyi bir seçimdir: pratik kural olarak, kilit iki bağlam değişiminden daha kısa süre tutulacaksa. Tek işlemcide dönmek anlamsızdır, çünkü biz dönerken kilidi tutan iş parçacığı çalışamaz.
- **Uyuma.** Bekleyen iş parçacığı bloklanır (bir bekleme kuyruğuna konur) ve işlemci başka bir iş parçacığını çalıştırır; `release` onu uyandırır. Uzun kritik bölgeler için bu daha iyidir. Linux pthread mutekslerini *futex*'lerle (fast user-space mutex, hızlı kullanıcı alanı muteksi) gerçekleştirir: boş bir kilidi almak tek bir atomik komuttur ve sistem çağrısı gerektirmez; yalnızca kilit meşgulse iş parçacığı çekirdekten kendisini uyutmasını ister.

12 Semaforlar

1965'te Edsger Dijkstra'nın ortaya koyduğu **semafor** (semaphore), ilk değer atama dışında yalnızca iki atomik işlemle erişilen bir S tamsayı değişkenidir:

```
wait(S) {            /* Dijkstra: P, Felemenkçe "proberen" (sınamak) */
    while (S <= 0)
        ;            /* meşgul bekleme */
    S--;
}

signal(S) {          /* Dijkstra: V, Felemenkçe "verhogen" (artırmak) */
    S++;
}
```

POSIX'te bu işlemlerin adları `sem_wait()` ve `sem_post()`'tur.

İkili ve sayan semaforlar.

- **İkili semafor** (binary semaphore) yalnızca 0 ve 1 değerlerini alır. 1 ile başlatılırsa bir muteks kilidi gibi davranır: kritik bölgeden önce `wait`, sonra `signal`.
- **Sayan semafor** (counting semaphore) negatif olmayan herhangi bir değeri alabilir. Özdeş kaynakların sayısı N ile (diyelim $N = 3$ yazıcı) başlatılırsa, kaynağı aynı anda en çok N iş parçacığının kullanmasına izin verir: her iş parçacığı bir yazıcıyı kullanmadan önce `wait`, sonra `signal` çağırır. Dördüncü iş parçacığı bir yazıcı boşalana kadar bloklanır.

Sıra dayatmak için semaforlar. Semaforlar bir sırayı da zorunlu kılabilir. B iş parçacığındaki S_2 deyiminin, ancak A iş parçacığındaki S_1 deyiminden sonra çalışması gerektiğini varsayalım. 0 ile başlatılmış bir `synch` semaforu kullanın: A iş parçacığı S_1; `signal(synch)` yürütür, B iş parçacığı ise `wait(synch)`; S_2. Zamanlayıcı ne yaparsa yapsın B, A bildirimde bulunmadan `wait`'i geçemez.

Meşgul beklemesiz gerçekleştirme. Yukarıdaki tanım döner. Gerçek gerçekleştirmeler her semafora bir *bekleme kuyruğu* ekler: `wait` boş kaynak bulamazsa iş parçacığı bloklanır ve kuyruğa konur; `signal` kuyruktan bir iş parçacığını çıkarır ve onu hazır hâle getirir. Bazı ders kitabı sürümleri S'nin negatif olmasına izin verir; bu durumda $-S$ bekleyen iş parçacıklarının sayısıdır. İşlemlerin kendileri bir önceki bölümün donanım komutlarıyla atomik yapılır; böylece biraz meşgul bekleme kalır, ama bütün kritik bölge için değil, yalnızca `wait` ve `signal`'in birkaç komutu için.

13 Sınırlı Tampon Problemi

Bir **üretici** (producer) iş parçacığı öğeler (örneğin çözülmüş video kareleri) üretir ve onları n gözlü bir tampona koyar; bir **tüketici** (consumer) iş parçacığı onları alır (ve gösterir). Tampon doluyken üretici, boşken tüketici beklemelidir; ikisi tampon indislerini asla aynı anda değiştirmemelidir. Üç semafor üretici–tüketici problemini çözer:

Semafor	Başlangıç değeri	Anlamı
`empty`	n	boş göz sayısı (sayan)
`full`	0	dolu göz sayısı (sayan)
`mutex`	1	tampon ve indisleri için kilit (ikili)

```
/* üretici */
while (true) {
    /* bir öğe üret */
    wait(empty);
    wait(mutex);
    /* öğeyi tampona ekle */
    signal(mutex);
    signal(full);
}
```

```
/* tüketici */
while (true) {
    wait(full);
    wait(mutex);
    /* öğeyi tampondan çıkar */
    signal(mutex);
    signal(empty);
    /* öğeyi tüket */
}
```

Roller simetriktir: üretici boş gözleri "tüketir" ve dolu gözler "üretir"; tüketici bunun tersini yapar. Tampon doluyken `empty` $= 0$'dır ve üretici, tüketici bildirimde bulunana kadar `wait(empty)` içinde bloklanır. İki `wait` çağrısının sırası önemlidir; bunu bir sonraki bölüm gösteriyor. POSIX semaforlarıyla gerçek bir C sürümü Bölüm 15.3'tedir.

14 Bir Uyarı: Kilitlenme

Kilitlerin dikkatsiz kullanımı iş parçacıklarının birbirini sonsuza dek beklemesine yol açabilir. S ve Q, 1 ile başlatılmış iki semafor olsun:

```
/* P0 iş parçacığı */
wait(S);
wait(Q);
  ...
signal(S);
signal(Q);
```

```
/* P1 iş parçacığı */
wait(Q);
wait(S);
  ...
signal(Q);
signal(S);
```

P0 `wait(S)` yürütür ve sonra kesilirse, P1 `wait(Q)` yürütebilir. Şimdi P0 `wait(Q)` içinde, P1 `wait(S)` içinde bloklanır: her biri yalnızca diğerinin yapabileceği bir `signal`'i bekler. Buna **kilitlenme** (deadlock) denir. En basit önleme kuralı, tüm iş parçacıklarının kilitleri aynı genel sırayla almasını sağlamaktır (burada: her zaman önce S, sonra Q).

Sınırlı tamponda da aynı tuzak vardır. Üretici tampon doluyken `wait(empty)`'den *önce* `wait(mutex)` çağırsaydı, `mutex`'i tutarken uyurdu; tüketici bir öğe çıkarmak için asla içeri giremez ve ikisi de sonsuza dek beklerdi. Açlıktan farkına dikkat edin: kilitlenmede gruptaki *hiç kimse* ilerlemez; açlıkta başkaları ilerler. Kilitlenmenin bütünüyle ele alınması (tespit, kaçınma, bankacı algoritması) bu dersin kapsamı dışındadır.

15 Uygulama: C'de İş Parçacıkları, Yarışlar ve Kilitler

Programlar `code/` klasöründedir; hepsini derlemek için `make` yazın (her biri `gcc -Wall -O2 -pthread` ile derlenir). POSIX threads kullanırlar: `pthread_create()` verilen bir fonksiyonu çalıştıran yeni bir iş parçacığı başlatır, `pthread_join()` onun bitmesini bekler. 2. haftada olduğu gibi C yazmanız beklenmiyor; açıklamaları okuyun ve ne olduğuna odaklanın.

15.1 Örnek 1: gerçek bir yarış durumu

```c
/* volatile: her artırmada gerçek bir okuma ve yazma yaptırır
 * (yoksa -O2 N'yi tek adımda ekler ve yarışı gizlerdi). */
volatile long counter = 0;       /* iki iş parçacığının ortak verisi */

void *worker(void *arg)
{
    for (long i = 0; i < N; i++)
        counter++;               /* oku, 1 ekle, yaz: atomik DEĞİL */
    return NULL;
}
int main(void)
{
    pthread_t a, b;
    double t0 = seconds();
    pthread_create(&a, NULL, worker, NULL);   /* A'yı başlat */
    pthread_create(&b, NULL, worker, NULL);   /* B'yi başlat */
    pthread_join(a, NULL);                    /* ikisini bekle */
    pthread_join(b, NULL);
    double t1 = seconds();

    printf("counter = %8ld of %ld  (lost %7ld)  %.3f s\n",
           counter, 2 * N, 2 * N - counter, t1 - t0);
    return 0;
}
```

Değişken `volatile` olarak bildirilmiştir; böylece derleyici onu her yinelemede gerçekten okuyup yazar; aksi hâlde eniyileyici döngüyü tek bir toplamayla değiştirip yarışı gizlerdi. Derleme makinemizde (2 işlemci) dört çalıştırma, ardından `taskset` ile tek bir işlemciye sabitlenmiş iki çalıştırma:

```
$ ./race          # run 4 times
counter = 12250428 of 20000000  (lost 7749572)  0.052 s
counter = 12005053 of 20000000  (lost 7994947)  0.100 s
counter = 13797565 of 20000000  (lost 6202435)  0.110 s
counter = 12046532 of 20000000  (lost 7953468)  0.080 s
$ taskset -c 0 ./race   # pinned to ONE CPU
counter = 14324099 of 20000000  (lost 5675901)  0.034 s
counter = 12867601 of 20000000  (lost 7132399)  0.034 s
```

Artırmaların %28 ile %40'ı kayboldu ve sonuç her seferinde farklıydı. İki işlemcide iş parçacıkları gerçekten aynı anda çalışır ve oku–ekle–yaz dizileri sürekli üst üste biner. Belki şaşırtıcı biçimde kayıplar *tek* işlemcide de büyük kalır. Nedeni şudur: bir iş parçacığı okuma ile yazma arasında kesildiğinde diğer iş parçacığı bütün bir zaman dilimi boyunca çalışır ve milyonlarca artırma yapar; ilk iş parçacığı kaldığı yerden devam ettiğinde eski, bayatlamış değerini yazar ve tüm o emeği tek bir komutla siler.

15.2 Örnek 2: muteksle çözüm

```c
void *worker(void *arg)
{
    for (long i = 0; i < N; i++) {
        pthread_mutex_lock(&lock);     /* al: giriş bölümü       */
        counter++;                     /* kritik bölge           */
        pthread_mutex_unlock(&lock);   /* bırak: çıkış bölümü    */
    }
```

```
    return NULL;
}
```

```
$ ./race_fixed    # run 3 times
counter = 20000000 of 20000000  (lost       0)  1.122 s
counter = 20000000 of 20000000  (lost       0)  1.115 s
counter = 20000000 of 20000000  (lost       0)  0.749 s
```

Sonuç artık her zaman doğrudur, ama program yaklaşık 10 ila 20 kat daha yavaştır (0.05–0.11 s yerine 0.75–1.12 s). Program 2×10^7 kilitle/aç çifti yürütür ve iki işlemci kilidi (ve onu tutan önbellek satırını) sürekli birbirine paslar. Kilitli artırma başına maliyet şudur:

$$\frac{0.749\ \text{s}}{2 \times 10^7} \approx 37\ \text{ns} \quad \text{ile} \quad \frac{1.122\ \text{s}}{2 \times 10^7} \approx 56\ \text{ns}.$$

Ders: önce doğruluk, sonra başarım. Daha iyi bir tasarım kritik bölgeye nadiren girer: her iş parçacığı özel bir yerel değişkende sayar ve toplamını `counter`'a kilit altında yalnızca bir kez ekler. O zaman yirmi milyon yerine yalnızca iki kilit işlemi gerekir.

15.3 Örnek 3: POSIX semaforlarıyla üretici ve tüketici

Üretici ve `main()`'in başı (tüketici, tıpkı sözde koddaki gibi üreticinin ayna görüntüsüdür; programın tamamı `code/prodcons.c` dosyasında):

```
void *producer(void *arg)
{
    for (int item = 1; item <= ITEMS; item++) {
        sem_wait(&empty);               /* boş göz bekle         */
        sem_wait(&mutex);               /* kritik bölgeye gir    */
        buffer[in] = item;
        in = (in + 1) % SIZE;
        sem_post(&mutex);               /* kritik bölgeden çık   */
        sem_post(&full);                /* bir dolu göz daha     */
    }
    return NULL;
}
int main(void)
{
    pthread_t p, c;
    sem_init(&empty, 0, SIZE);   /* 0 = süreçler değil, iş parçacıkları paylaşır */
    sem_init(&full,  0, 0);
    sem_init(&mutex, 0, 1);
    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);
```

```
$ ./prodcons
consumed  5   running sum  15
consumed 10   running sum  55
consumed 15   running sum 120
consumed 20   running sum 210
sum of 1..20 = 210 (expected 210)
```

$1 + 2 + \cdots + 20 = 20 \cdot 21/2 = 210$ toplamı, tamponun yalnızca 5 gözü olmasına ve üreticinin 5 öğe öne

geçtiğinde tekrar tekrar bloklanmasına karşın her öğenin tam bir kez ulaştığını gösterir: hiçbiri kaybolmadı ya da yinelenmedi. `sem_init()`'in ikinci argümanı olan 0, semaforun tek bir sürecin iş parçacıkları arasında paylaşıldığı anlamına gelir (sıfırdan farklı bir değer, ortak bellek üzerinden süreçler arasında paylaşıma izin verirdi).

16 Özet

Yedi cümlede 5. hafta

1. FCFS basittir ama konvoy etkisinden muzdariptir: aynı üç iş, yalnızca sıralarına bağlı olarak 17 ya da 3 ms ortalama bekleme verir.
2. SJF, tüm işler birlikte hazırken; SRTF, işler zamana yayılarak gelirken ortalama bekleme süresini en aza indirir; ikisi de $\tau_{n+1} = \alpha t_n + (1-\alpha)\tau_n$ ile tahmin edilen bir sonraki patlama uzunluğuna ihtiyaç duyar.
3. Öncelikli zamanlama düşük öncelikli süreçleri aç bırakabilir; yaşlandırma onların önceliğini zamanla yükseltir.
4. Döngüsel zamanlama her beklemeyi $(n-1)q$ ile sınırlar; çok büyük bir q FCFS verir, çok küçük bir q işlemciyi geçişlerle israf eder. Linux normal görevleri nice değerleriyle ağırlıklandırılmış adalete göre zamanlar (CFS, 6.6'dan beri EEVDF).
5. Eşzamanlı iş parçacıkları ortak veriye eriştiğinde, en az biri yazdığında ve sonuç zamanlamaya bağlı olduğunda bir yarış durumu doğar; `counter++` oku–ekle–yaz demektir ve güncelleme kaybettirebilir.
6. Kritik bölge probleminin bir çözümü karşılıklı dışlamayı, ilerlemeyi ve sınırlı beklemeyi garanti etmelidir; atomik donanım komutları muteks kilitlerinin ve semaforların temelidir.
7. Semaforlar sınırlı tampon problemini çözer (`empty`, `full`, `mutex`), ama tutarsız sıralarla alınan kilitler kilitlenmeye yol açabilir.

17 Sözlük

Açlık (starvation) Hazır bir sürecin ihtiyaç duyduğu kaynağı hiçbir zaman alamaması.

Karşılıklı dışlama (mutual exclusion) Herhangi bir anda kritik bölgesini en çok bir iş parçacığının yürütmesi.

Kilitlenme (deadlock) Bir gruptaki her iş parçacığının, yalnızca gruptaki başka bir iş parçacığının tetikleyebileceği bir olayı beklediği durum.

Kritik bölge (critical section) Bir iş parçacığının ortak veriye eriştiği kod.

Muteks kilidi (mutex lock) Karşılıklı dışlama sağlayan, `acquire` ve `release` işlemli bir kilit.

Semafor (semaphore) Yalnızca atomik `wait` (P) ve `signal` (V) işlemleriyle erişilen bir tamsayı.

Sınırlı bekleme (bounded waiting) Kritik bölgesine girmeyi bekleyen bir iş parçacığının ancak sınırlı sayıda geçilmesi gereksinimi.

Yarış durumu (race condition) Ortak veriye eşzamanlı erişimlerin sonucunun zamanlamalarına bağlı olduğu

durum.

Yaşlandırma (aging) Açlığı önlemek için uzun süre beklemiş bir sürecin önceliğini kademeli olarak yükseltmek.

Zaman dilimi (time quantum) Döngüsel zamanlamada bir sürecin tek seferde en çok çalışabileceği süre.

18 Kendiniz Deneyin

Kendi Linux makinenizde yarışlar ve zamanlayıcılar

`code/` klasöründe (laboratuvar bilgisayarları, WSL ya da bir sanal makine):

```
$ make                      # race, race_fixed, prodcons, cpu_hog derlenir
$ ./race; ./race; ./race    # her seferinde farklı bir yanlış yanıt mı?
$ time ./race_fixed         # doğru, ama ne kadar yavaş?
$ python3 sched_check.py    # bu notlardaki tüm Gantt şemaları
```

1. `race` sizin makinenizde kaç güncelleme kaybediyor? Onu tek bir işlemciye sabitlemek (`taskset -c 0 ./race`) tabloyu değiştiriyor mu?
2. `race_fixed.c`'yi her iş parçacığı yerel bir değişkende sayacak ve toplamı `counter`'a kilit altında bir kez ekleyecek biçimde değiştirin. Şimdi ne kadar hızlı? Hâlâ doğru mu?
3. `prodcons.c`'de üreticideki iki `sem_wait()` çağrısının yerini değiştirin ve `SIZE`'ı 1 yapın. Ne oluyor ve neden?

19 Tekrar Soruları

Kavramsal

1. Tüm işler birlikte geldiğinde en kısa iş önce zamanlamasının ortalama bekleme süresini en aza indirdiğini yer değiştirme argümanıyla kanıtlayın.
2. SJF bir işlemci zamanlayıcısında neden tam olarak gerçekleştirilemez? İşletim sistemi ona nasıl yaklaşır? $\alpha = 0$ ve $\alpha = 1$ iken tahmine ne olur?
3. Açlık nedir? Hangi algoritmalar açlığa yol açabilir? Yaşlandırma onu nasıl çözer?
4. Yarış durumunun ne olduğunu kendi sözcüklerinizle açıklayın. İki iş parçacığı `counter++` işlemini eşzamanlı yürüttüğünde neden yanlış sonuç çıkar?
5. Kritik bölge probleminin bir çözümü için üç gereksinimi ifade edin. Karşılıklı dışlamayı sağlayan ama ilerlemeyi sağlamayan bir "çözüm" verin.
6. Bir döner kilidi ve uyuyan bir muteksi karşılaştırın. Her biri ne zaman daha iyi bir seçimdir?
7. İkili ve sayan semafor arasındaki fark nedir? Her birine bir kullanım örneği verin.
8. Sınırlı tampon çözümünde `empty`, `full` ve `mutex` semaforlarının her biri neyi sayar ya da korur? Üretici `mutex`'i `empty`'den önce alırsa ne ters gider?

Sayısal

9. Dört süreç şöyle geliyor (süreler ms): A (0, patlama 6), B (1, patlama 3), C (2, patlama 8), D (3, patlama 2). (a) FCFS, (b) kesintisiz SJF, (c) SRTF, (d) $q = 3$ ile RR için Gantt şemasını çizin ve ortalama bekleme süresini hesaplayın (eşzamanlı olaylar için kuralımızı kullanın).
10. $\alpha = 0.5$ ve $\tau_0 = 12$ ms ile üstel ortalama kullanılıyor; bir sürecin ilk üç patlaması 10, 6 ve 8 ms. τ_1, τ_2 ve τ_3'ü hesaplayın.
11. Bir bağlam değişimi $0{,}5$ ms, zaman dilimi 20 ms sürüyor. Tüm süreçler zaman dilimlerinin tamamını kullandığında işlemci zamanının ne kadarı geçişe gider?
12. `race_fixed` 2×10^7 kilitli artırmayı $0{,}749$ s'de, `race` ise aynı sayıda kilitsiz artırmayı $0{,}052$ s'de yaptı. Her durumda artırma başına süreyi ve yavaşlama çarpanını hesaplayın.

Sayısal soruların yanıtları

9. (a) FCFS: A 0–6, B 6–9, C 9–17, D 17–19; beklemeler $0, 5, 7, 14$; ortalama $26/4 = 6.5$ ms. (b) SJF: A 0–6, D 6–8, B 8–11, C 11–19; beklemeler $0, 7, 9, 3$ (A, B, C, D); ortalama $19/4 = 4.75$ ms. (c) SRTF: A 0–1, B 1–4, D 4–6, A 6–11, C 11–19; beklemeler A $= 11 - 6 = 5$, B $= 0$, C $= 9$, D $= 1$; ortalama $15/4 = 3.75$ ms. (d) RR, $q = 3$: A 0–3, B 3–6, C 6–9, D 9–11, A 11–14, C 14–19; beklemeler A $= 8$, B $= 2$, C $= 9$, D $= 6$; ortalama $25/4 = 6.25$ ms.
10. $\tau_1 = 0.5 \cdot 10 + 0.5 \cdot 12 = 11$ ms; $\tau_2 = 0.5 \cdot 6 + 0.5 \cdot 11 = 8.5$ ms; $\tau_3 = 0.5 \cdot 8 + 0.5 \cdot 8.5 = 8.25$ ms.
11. $s/(q + s) = 0.5/20.5 \approx 0.024$, yani yaklaşık %2,4.
12. Kilitli: $0.749/(2 \times 10^7) \approx 37$ ns; kilitsiz: $0.052/(2 \times 10^7) = 2.6$ ns; yavaşlama $\approx 37.45/2.6 \approx 14$. (Üstelik kilitsiz sürüm yanlıştır.)

20 İleriye Bakış: 6. Hafta

- İşlemciden ikinci büyük kaynağa: ana bellek. Adres bağlama; mantıksal ve fiziksel adresler; bellek yönetim birimi (memory-management unit, MMU).
- Takas (swapping) ve bitişik bellek tahsisi; ilk uyan (first fit), en iyi uyan (best fit), en kötü uyan (worst fit).
- Dış ve iç parçalanma (fragmentation). Sayfalama: sayfalar, çerçeveler ve sayfa tabloları; TLB ve etkin erişim süresi; bölütleme (segmentation). (Lütfen ikinin kuvvetlerini gözden geçirin: $2^{10} = 1024$, $2^{12} = 4096$.)

İleri okuma (isteğe bağlı)

- A. Silberschatz, P. B. Galvin ve G. Gagne, *Operating System Concepts*, 10. baskı, Wiley: Bölüm 5 (CPU Scheduling), Bölüm 6 (Synchronization Tools), Kısım 7.1 (sınırlı tampon); kilitlenme Bölüm 8'de.
- R. H. ve A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, bölüm PDF'leri:
  - Bölüm 7, Scheduling: Introduction: `https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-sched.pdf`
  - Bölüm 8, Multi-Level Feedback Queue: `https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-sched-mlfq.pdf`

- Bölüm 26, Concurrency: An Introduction: https://pages.cs.wisc.edu/~remzi/OSTEP/threads-intro.pdf
- Bölüm 28, Locks: https://pages.cs.wisc.edu/~remzi/OSTEP/threads-locks.pdf
- Bölüm 31, Semaphores: https://pages.cs.wisc.edu/~remzi/OSTEP/threads-sema.pdf